

Task 2 – LLM-Use Report

COMP5111 Assignment 2

ZIMO JI, HKUST, Hong Kong SAR

1 The LLM I used

I used **ChatGPT (GPT-5.4)** through an OpenAI-compatible chat endpoint. I did not let it run my pipeline; I pasted relevant rows of the *.tsv spectra, the failing test bodies, and chunks of Subject.java into the chat and asked specific questions about suspicious Jimple statements.

2 Where the LLM helped

2.1 Picking the right row when one Jimple stmt maps to two source lines

In `parseToken` the buggy `pos++`; (line 205) and the harmless `pos++`; (line 208) both render as `i2 = i2 + 1`. My TSV has two rows with that text, scored 0.617 and 0.552 (in `randoop0`), but no source line. I asked the LLM which one was likely the buggy one. It pointed out that line 205 only fires when a terminator is hit, which is exactly the path that fails, so it should score higher. That matched my data and let me cite `rank=7, score=0.617` in `fault_205.txt` without guessing.

2.2 One-line specs for the suspicious methods

Reading the Javadocs of `parseToken`, `extractIntInStr`, `daysBetweenDateStrings`, and `monAbbr2month` is slow. I asked the LLM to summarise each contract in one sentence. For `extractIntInStr` it gave me “return the LAST contiguous digit run, e.g. `1234a123` → `123`”, and once I had that, the failing test `extractIntInStr("0366-01-31")` expected `31` made the bug obvious: the code returns at the first non-digit, but the spec wants the last digit run. I had read the function a few times and convinced myself it “returns the leading int” before the LLM made me look at the Javadoc again.

2.3 Computing the 12 perfect-hash constants for `monAbbr2month`

The method dispatches on `(ch0 « 16) | (ch1 « 8) | ch2` against 12 hard-coded integers. I asked the LLM to compute the hashes for “Jan” through “Dec” and check them against the source. It returned a table where every row matched except Sep: source 5465466, computed 5465456. I confirmed by hand: `(‘S’ « 16) | (‘e’ « 8) | ‘p’ = 5439488 + 25856 + 112 = 5465456`. That is exactly the off-by-10 fault on line 615. Doing this without the LLM is 12 multiplications and additions, all correct, which I would not have trusted myself to do without a typo.

2.4 Splitting “Calendar bug” from “validity bug” in `daysBetweenDates`

The failing tests `daysBetweenDateStrings("0100-10-31", "0366-01-31")` etc. light up both `daysBetweenDates` (lines 442–454, the Calendar code) and `checkValidDate` (line 488). I asked the LLM where in `daysBetweenDateStrings` the value `Integer.MIN_VALUE` could come from. It pointed out that `MIN_VALUE` only comes from the catch block, so an exception must be thrown before the Calendar arithmetic runs. With `day=31` in the failing inputs, the only exception

source is `checkValidDate` rejecting `day=31` of a 31-day month. That moved my attention straight to line 488 and I stopped looking at the `Calendar` code.

3 What the LLM was not useful for

- It could not improve the ranking. Ochiai already gives the right ordering up to ties; the LLM does not change the score formula.
- It could not, on its own, separate two source lines that share the same Jimple text. I had to feed it the metadata file mapping stmts to lines.
- It kept proposing rewrites for `extractIntInStr` (e.g. “use a regex”), which violates the single-line patch rule. I had to remind it of the constraint.

4 Net

I used the LLM as a Javadoc reader and an arithmetic checker on top of my Ochiai shortlist. That cut the manual inspection from “every row in top-30” to “the four lines I am submitting as faults”. After applying my four single-line patches, all 1371 tests in the four provided suites pass.

(Word count, excluding code: about 470.)